

Как добавить новую страницу в apps/demo

От Figma-макета до запущенной страницы — 3 команды.

Полные правила и обоснования — в [AGENTS.md](#). Машинно-читаемые правила (для slash-команд) — в [.claude/page-recipe.yaml](#). При расхождении приоритет у `AGENTS.md`.

TL;DR — три команды

Когда	Команда	Что происходит
Один раз на машине	<code>pnpm rollout:setup</code>	<code>nvm</code> + Node 22.21.1 + <code>pnpm</code> 10.32.1 + <code>pnpm install</code> + <code>preflight</code>
Один раз на машине	установить 3 MCP-плагина (см. §1.5)	Framelink Figma + Shadcn_UI + Claude_Preview — без них <code>/new-page</code> молча умрёт
После <code>setup</code> обязательно, далее при сомнениях	<code>pnpm rollout:preflight</code> + <code>/preflight</code> в Claude Code	bash- и MCP-сторона диагностики
На каждую страницу	<code>/new-page</code> в Claude Code	агент задаёт 5 вопросов и сам делает всё от Figma до <code>preview-screenshot</code>

`/new-page` без Claude Code — есть `pnpm rollout:new-page`: те же 5 вопросов, готовый промпт кладётся в clipboard, вставляешь в любой LLM-агент.

1.0 Предусловия — Homebrew и git CLI (до клонирования)

Без `git` команда `git clone` ниже не работает. На macOS — самый простой путь через Homebrew.

Шаг 1. Homebrew (только macOS, один раз на машине)

Проверь, установлен ли он:

```
brew --version
```

Если вывод вида `Homebrew 4.x.x` — пропусти этот шаг. Если команда не найдена — открой **Terminal.app** (Cmd+Space → «Terminal») и выполни официальную команду установки:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Установщик спросит пароль (`sudo`) и сам поставит Xcode Command Line Tools, если их нет. В конце он напечатает 2–3 строки «**Next steps**» — это команды вида `echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile`, которые нужно скопировать и выполнить, чтобы `brew` появился в PATH в новых терминалах. Затем перезапусти терминал и снова проверь `brew --version`.

Документация: brew.sh.

Шаг 2. git CLI

```
git --version
```

Если вывод вида `git version 2.x.x` — пропусти этот шаг. Если команда не найдена — поставь по своей платформе:

Платформа	Команда
macOS (Homebrew)	<code>brew install git</code> (рекомендуется — свежая версия, авто-обновления через <i>brew upgrade</i>)
macOS (Xcode CLI)	<code>xcode-select --install</code> — поднимет окно установки Command Line Tools, внутри есть git, но версии обновляются медленнее
Ubuntu / Debian	<code>sudo apt update && sudo apt install -y git</code>
Fedora / RHEL	<code>sudo dnf install -y git</code>
Arch	<code>sudo pacman -S git</code>
Windows	git-scm.com/download/win — установщик Git for Windows

После установки повтори `git --version` — должна вывестись версия. Только после этого переходи к §1.

1. Установка (один раз)

```
git clone https://github.com/rolloutrf/rollout-ui.git ~/rollout-ui
cd ~/rollout-ui
pnpm rollout:setup
```

Скрипт сам поставит nvm-managed Node 22.21.1, активирует pnpm через corepack, выполнит `pnpm install`, в конце прогонит preflight. Если nvm не установлен — напечатает строку `curl ... install.sh` и остановится; запусти её и повтори.

Дополнительно — Claude Code: <https://docs.claude.com/en/docs/claude-code>.

В `~/.claude/launch.json` нужна запись `rollout-ui-demo` — без неё `mcp__Claude_Preview__preview_start({name:'rollout-ui-demo'})` не найдёт проект. Вставь блок ниже в массив `configurations[]` (замени `<repo>` и `<HOME>` на свои абсолютные пути):

```
{
  "name": "rollout-ui-demo",
  "cwd": "<repo>/apps/demo",
  "runtimeExecutable": "<HOME>/nvm/versions/node/v22.21.1/bin/node",
  "runtimeArgs": ["./.node_modules/vite/bin/vite.js"],
```

```
"port": 5173
}
```

`pnpm rollout:preflight` сам напечатает этот блок с подставленными путями и скопирует в `clipboard ( pbcopy )`.

1.5 Claude Code MCP-плагины (один раз)

`/new-page` и `/update-page` зависят от **трёх** MCP-серверов в Claude Code. `pnpm rollout:setup` их не ставит — это секреты и конфигурация пользователя. Установи через Claude Code (Settings → MCP / Plugins):

Плагин	Имя тулов	Зачем
Framelink Figma	<code>mcp__figma__*</code>	читать узлы Figma и забирать PNG для diff. Нужен токен — см. §1.5.1 ниже.
Shadcn_UI	<code>mcp__Shadcn_UI__*</code>	эталон поведения и визуала компонентов
Claude_Preview	<code>mcp__Claude_Preview__*</code>	dev-сервер, скриншоты, console-logs, resize для верификации

Code Connect MCP (`mcp__c0861a9b-...`) **не нужен** — see [AGENTS.md §0.3](#).

1.5.1 Framelink Figma MCP — установка и токен

Это самый частый blocker на свежей машине, поэтому пошагово:

Шаг 1. Получи Personal Access Token в Figma.

- 1. Залогинься на [figma.com](#).
- 2. Открой [figma.com/developers/api#access-tokens](#) → кнопка «**Get personal access token**» (или: Settings → Security → Personal access tokens → Generate new token).
- 3. Имя токена — любое (например, `rollout-ui-mcp`). Срок — на твой выбор.
- 4. Скоупы: достаточно **File content** → **Read-only**. Остальные не нужны.
- 5. Нажми **Generate token** и **сразу скопируй** значение (`figd_...`) — Figma больше его не покажет. Если потерял — придётся выпускать новый.

Шаг 2. Установи Framelink Figma MCP в Claude Code.

Удобнее всего через CLI Claude Code (он сам пропишет блок в `~/.claude.json`):

```
claude mcp add figma -- npx -y figma-developer-mcp --stdio
```

После этого открой `~/.claude.json` и в блок `mcpServers.figma` добавь `env` с токеном (если его там ещё нет):

```
"figma": {
  "command": "npx",
  "args": ["-y", "figma-developer-mcp", "--stdio"],
  "env": {
    "FIGMA_API_KEY": "figd_XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
  }
}
```

```
}  
}
```

Шаг 3. Перезапусти Claude Code (Cmd+Q → запуск заново). `env` читается ровно один раз при старте процесса; правка `~/ .claude.json` без рестарта не подхватится — это причина 90% «у меня figma 403 / не отвечает».

Шаг 4. Проверь. В новой сессии Claude Code выполни `/preflight` — строка `figma` должна быть зелёной. Если 403 — токен невалидный или просрочен, выпусти новый и повтори Шаг 2 (`env`) + Шаг 3 (рестарт).

Альтернатива через UI: Settings → MCP / Plugins → Add → выбрать «Framelink Figma» из реестра, вставить токен в поле `FIGMA_API_KEY`. Если такого плагина нет в списке твоей сборки Claude Code — используй CLI выше, результат идентичный.

Проверь, что **все три** MCP подключены, командой `/preflight` (см. §2). До зелёного `/preflight` запускать `/new-page` бессмысленно — он остановится на Step 0.

1.6 Глобальные slash-команды (опционально, один раз)

По умолчанию `/new-page`, `/update-page`, `/preflight` и `/setup` живут в `~/rollout-ui/.claude/commands/` и доступны только когда Claude Code запущен **из корня репо** (`cd ~/rollout-ui && claude`). Если ты часто работаешь в других папках и хочешь, чтобы команды были глобальными — создай в `~/ .claude/commands/` тонкие обёртки, которые сначала делают `cd ~/rollout-ui`, а потом делегируют исполнение оригинальной команде из репо.

Команды-обёртки (можно создать вручную или попросить Claude Code один раз):

Имя глобальной команды	Что делает
<code>/new-page</code>	<code>cd ~/rollout-ui</code> → читает и выполняет <code>~/rollout-ui/.claude/commands/new-page.md</code>
<code>/update-page</code>	то же, но для <code>update-page.md</code>
<code>/preflight</code>	то же, но для <code>preflight.md</code>
<code>/rollout-setup</code>	то же, но для <code>setup.md</code> . Важно: имя <code>/setup</code> зарезервировано Claude Code (auto-mode classifier блокирует запись в <code>~/ .claude/commands/setup.md</code>), поэтому глобальная обёртка называется <code>/rollout-setup</code> . Внутри репо команда по-прежнему доступна как <code>/setup</code> .

Минимальный шаблон обёртки (например, `~/ .claude/commands/new-page.md`):

```
---  
description: Create a new page in apps/demo (auto-cd to ~/rollout-ui)  
---  
  
**Step -1.** Run via Bash: `cd ~/rollout-ui && pwd`. Если репо нет — HARD STOP с инструкцией `git clone https://github.com/rolloutrf/rollout-ui.git ~/rollout-ui`. Используй `~/rollout-ui` как рабочую директорию для всех последующих вызовов.
```

****Step 0+.**** Прочитай и выполни инструкции из ``~/rollout-ui/.claude/commands/new-page.md`` дословно.

Плюс этого подхода: оригинал команды в репо остаётся источником истины, обёртка ничего не дублирует — после `git pull` глобальная команда сразу подхватит обновлённую версию.

⚠ Обязательная перезагрузка Claude Code

После создания (или правки) файлов в `~/ .claude/commands/` Claude Code **не перечитывает их в горячем режиме**. Чтобы команда `/new-page` появилась в списке slash-команд из любой директории — **полностью перезапусти Claude Code**:

- 1. `Cmd+Q` (macOS) или закрой окно полностью.
- 2. Запусти `claude` заново.
- 3. Проверь, что команды видны: набери `/` и в выпадающем списке должны быть `new-page` , `update-page` , `preflight` , `rollout-setup` .

Без рестарта команда не появится, и `/new-page` будет ругаться «unknown command». Это самая частая причина «у меня обёртки созданы, а не работают».

2. Pre-flight (обязательно после setup, потом — при сомнениях)

```
pnpm rollout:preflight
```

Зелёный вывод = можно работать. Если что-то красное — скрипт печатает команду, которой это лечится. Подводные камни (Node 20 без rolldown-биндинга, отсутствие конфига preview, неактивный npm) лечатся автоматически или одной строкой.

В Claude Code обязательно запусти `/preflight` хотя бы один раз после `pnpm rollout:setup` — он добавляет три MCP-проверки (Figma, Shadcn_UI, Claude_Preview), которые из bash не сделать. Без зелёного `/preflight` `/new-page` остановится на Step 0 (MCP precheck). Code Connect MCP намеренно не пробится — см. [AGENTS.md §0.3](#).

3. Новая страница — `/new-page`

Открой Claude Code в `~/rollout-ui` , введи `/new-page` . Агент спросит пять вещей и сам сделает всё остальное:

Вопрос	Пример
Название экрана	Личные данные
Figma URL (с ?node-id=...)	https://www.figma.com/design/p2bAIyTB6oJTGwjR8NwRB/Demo-App?node-id=221-4087
Откуда переход (файл страницы)	apps/demo/src/pages/profile/ProfilePage.tsx
На каком элементе	блок «Аккаунт» — секция с аватаром и email
Новый route	/profile/personal-data

Описывай **«элемент»** через визуальные приметы и маркер-текст (`<p>Аккаунт</p>` , аватар, email), а не «строки 103–115». Так промпт переживёт правки файла.

Альтернатива без Claude Code:

```
pnpm rollout:new-page # 5 интерактивных вопросов
pnpm rollout:new-page "Личные данные" \
  "https://figma..." \
  "apps/demo/src/pages/profile/ProfilePage.tsx" \
  "блок «Аккаунт»" \
  "/profile/personal-data" # argv-driven
```

Скрипт собирает промпт по шаблону и кладёт в clipboard (`pbcopy`). Лог последнего промпта — `~/rollout-ui/.claude/last-prompt.md` .

Аналог для уже существующих страниц — `/update-page` (или `pnpm rollout:update-page <file> <figma-url>`): два вопроса, агент делает diff с макетом и приводит код в соответствие.

Что агент делает за тебя (для понимания score)

Не нужно делать руками — это случится в `/new-page` :

1. Параллельно читает Figma — `mcp__figma__get_figma_data` (структура+токены) + `mcp__figma__download_figma_images` (PNG в `.tmp/figma-ref/`). Code Connect MCP не дёргается — см. [AGENTS.md §0.3](#).
2. Сверяет компоненты со shadcn/ui MCP (эталон поведения и визуала).
3. Создаёт `apps/demo/src/pages/<area>/<Name>Page.tsx` с контейнером `max-w-[576px] mx-auto pt-20 pb-8 gap-7` (без `px-*` — `edge-to-edge`).
4. NavBar по макету, токены — только переменные (Geist, `bg-accent / bg-muted / text-foreground` , без хексов).
5. Реализация компонентов на `@base-ui/react` через `@rollout/ui-kit` (HE Radix, HE `pnpm dlx shadcn add`).
6. Добавляет `<Route>` в `apps/demo/src/App.tsx` и заменяет элемент входа на `<Link to>` (НЕ `<a href>`).
7. Верифицирует через `Claude_Preview` (light + dark + mobile, console errors empty), сравнивает со скриншотом из Figma.
8. Обновляет дерево в [AGENTS.md §2](#) и таблицу Routing если изменился `App.tsx` .

Полный набор правил, которые применяет агент, — в [.claude/page-recipe.yaml](#). Если что-то нужно изменить (например, новый запрет или другой контейнер) — правь yaml, slash-команды подхватят без правок промптов.

Чек-лист (перед PR)

- ☐ Pre-flight зелёный (`pnpm rollout:preflight`)
- ☐ `preview_screenshot` ↔ `.tmp/figma-ref/<slug>.png` совпадают (light + dark)
- ☐ `preview_console_logs level: 'error'` пусто
- ☐ Нет хардкоднутых цветов/шрифтов/размеров
- ☐ Нет deep imports, `radix-ui` , `pnpm dlx shadcn add`

- ☐ Дерево в [AGENTS.md §2](#) обновлено
- ☐ При новом ui-kit компоненте — добавлен `pnpm changeset` (minor для `@rollout/ui-kit`)
- ☐ Если страница со своим scroll-контейнером — `pb-tabbar md:pb-0` повешен

Полезные ссылки

- Полные правила: [apps/demo/AGENTS.md](#)
- Корневые правила монорепы: [~/rollout-ui/AGENTS.md](#)
- Машинные правила (для slash-команд): [.claude/page-recipe.yaml](#)
- Storybook (live): <https://rolloutrf.github.io/rollout-ui>
- shadcn Base UI reference: <https://ui.shadcn.com/docs/components/base/select>
- Base UI docs: <https://base-ui.com/react/components>
- Figma file keys:
 - Demo App (макеты экранов): `p2bAIyTB6oJTGWjjR8NwRB`
 - shadcn · actual (эталонная DS): `Rf9NPBgJ0gcj504cSoo8kg`
- Контакт: Telegram `@rolloutrf` — для обсуждения больших фич перед PR

Частые проблемы

Симптом	Лечение
Cannot find native binding	<code>pnpm rollout:preflight</code> (с <code>ROLLOUT_AUTOFIX=1</code> сам переустановит <code>node_modules</code> под Node 22)
<code>getComputedStyle(body).fontFamily</code> не Geist	проверь <code>html { font-family: var(--font-sans); }</code> в <code>index.css</code> (см. AGENTS.md §3)
Заголовок страницы скрыт за Header	контейнеру добавить <code>pt-20</code> (компенсирует ~72px fixed Header)
Highlighted в Select оранжевый	заменить <code>bg-accent</code> на <code>data-highlighted:bg-muted data-highlighted:text-foreground</code>
Figma MCP даёт 403	новый ключ на figma.com/developers/api , правка <code>~/claude.json</code> → <code>mcpServers.figma.env.FIGMA_API_KEY</code> , рестарт Claude Code (env читается на старте)
Preview подхватывает чужой проект	имя должно быть <code>rollout-ui-demo</code> , не <code>rollout-demo</code> ; перепроверь <code>cwd</code> в <code>~/claude/launch.json</code>
<code>pnpm dev</code> нет такого скрипта	используй <code>pnpm --filter @rollout/demo dev</code> или MCP <code>preview_start({name: 'rollout-ui-demo'})</code>

Если нужно посмотреть «как это работало раньше» (длинный *manual prompt-template* из §2, пошаговое объяснение Figma → shadcn → page → routing → verify) — история живёт в git: `git log -p apps/demo/HOW_TO_ADD_PAGE.md`. Сегодня всё это — работа агента, не пользователя.

Если что-то не по списку — Telegram `@rolloutrf`, ДО того, как тратить часы на угадывание.